

1.7. Гауссовские процессы

Гауссовы процессы (Gaussian Processes - GP) - это непараметрический метод контролируемого обучения, используемый для решения задач *регрессии* и *вероятностной классификации*.

Преимуществами гауссовских процессов являются:

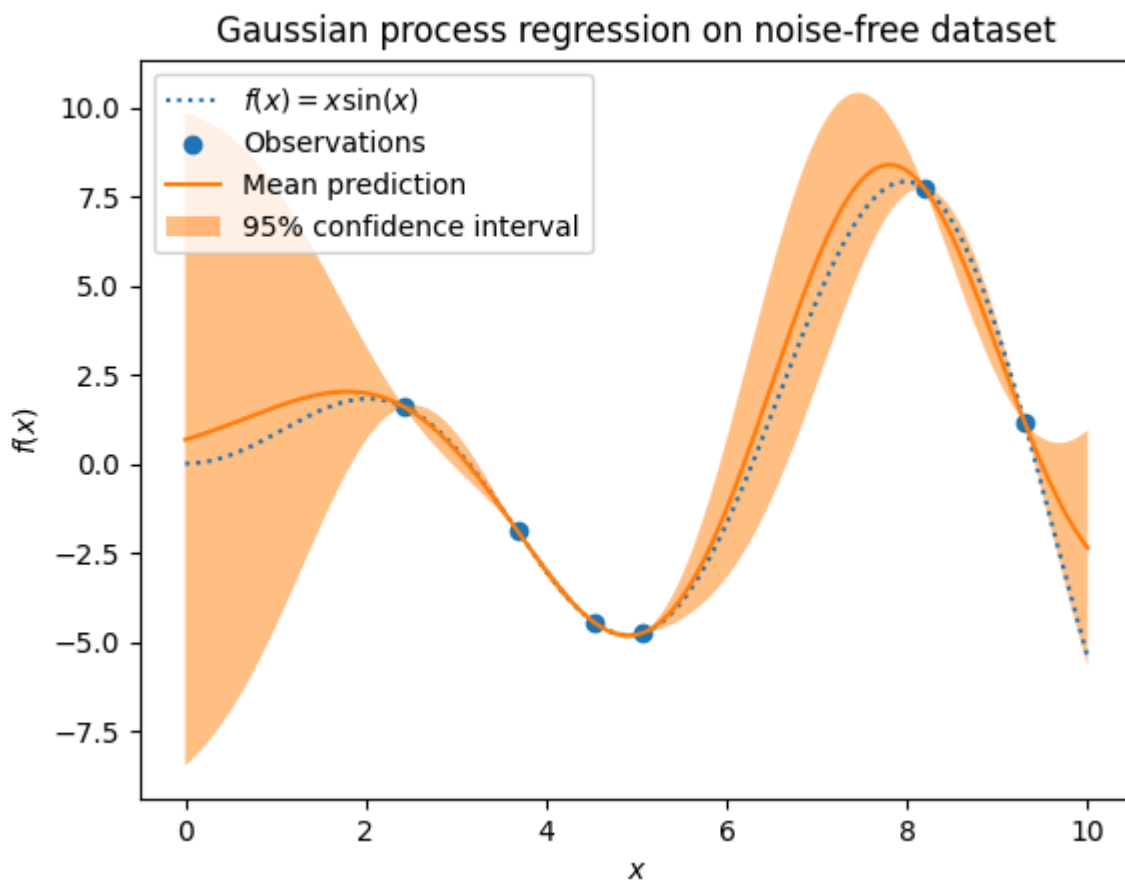
- Прогноз интерполирует наблюдения (по крайней мере, для регулярных ядер).
- Прогноз является вероятностным (гауссовским), так что можно вычислить эмпирические доверительные интервалы и на их основе решить, нужно ли обучать (онлайн обучение, адаптивное обучение) прогноз в некоторой интересующей области.
- Универсальность: можно указать различные [ядра](#). Предоставляются общие ядра, но можно указать и собственные.

К недостаткам гауссовских процессов относятся:

- Наша реализация не является разреженной, т.е. они используют всю информацию о выборках/функциях для предсказания.
- Они теряют эффективность в пространствах высокой размерности - а именно, когда число признаков превышает несколько десятков.

1.7.1. Регрессия с использованием гауссовых процессов (Gaussian Process Regression - GPR)

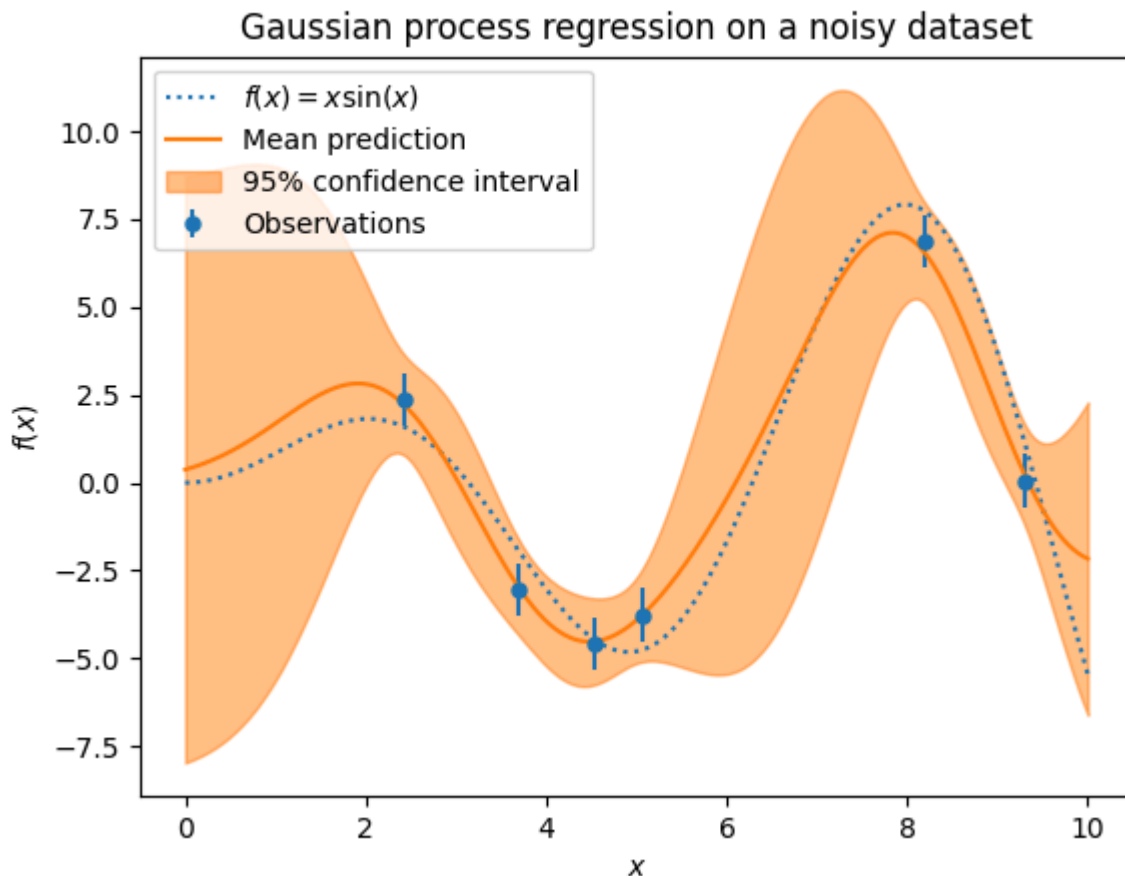
[GaussianProcessRegressor](#) реализует гауссовские процессы для целей регрессии. Для этого необходимо указать приоритет GP. GP объединит это предшествование и функцию правдоподобия, основанную на обучающих выборках. Это позволяет использовать вероятностный подход к прогнозированию, выдавая среднее и стандартное отклонение в качестве выходных данных при прогнозировании.



Предполагается, что предварительное среднее является постоянным и равно нулю (для `normalize_y=False`) или среднему значению обучающих данных (для `normalize_y=True`). Априорная ковариация задается путем передачи объекта [ядрва](#). Гиперпараметры ядра оптимизируются при обучении [GaussianProcessRegressor](#) путем максимизации лог-маргинального правдоподобия (log-marginal-likelihood - LML) на основе переданного [оптимизатора](#). Поскольку LML может иметь несколько локальных оптимумов, оптимизатор можно запускать многократно, указав `n_restarts_optimizer`. Первый запуск всегда выполняется с начальными значениями гиперпараметров ядра; последующие запуски выполняются со значениями гиперпараметров, выбранными случайным образом из диапазона допустимых значений. Если начальные гиперпараметры должны быть фиксированными, в качестве оптимизатора можно передать `None`.

Уровень шума в целях может быть задан путем передачи его через параметр `alpha`, либо глобально как скаляр, либо для каждой точки данных. Обратите внимание, что умеренный уровень шума может также помочь справиться с неустойчивостью чисел во время обучения, поскольку он эффективно реализуется как регуляризация Тихонова, т.е. путем добавления к диагонали матрицы ядра. Альтернативой явному указанию уровня шума является включение в ядро компонента [WhiteKernel](#), который может оценить

глобальный уровень шума по данным (см. пример ниже). На рисунке ниже показан эффект обработки зашумленной цели с помощью параметра `alpha`.



Реализация основана на алгоритме 2.1 из [\[RW2006\]](#). В дополнение к API стандартных моделей scikit-learn, [GaussianProcessRegressor](#) :

- позволяет предсказывать без предварительного обучения (на основе GP-приор)
- предоставляет дополнительный метод `sample_y(X)`, который оценивает образцы, взятые из GPR (предшествующие или последующие) при заданных входных данных
- раскрывает метод `log_marginal_likelihood(theta)`, который можно использовать извне для других способов выбора гиперпараметров, например, с помощью цепи Маркова Монте-Карло.

1.7.2. Классификация на основе гауссовского процесса (Gaussian Process Classification - GPC)

[GaussianProcessClassifier](#) реализует гауссовские процессы для целей классификации, а точнее для вероятностной классификации, где предсказания теста имеют форму вероятностей классов.

GaussianProcessClassifier накладывает GP-приор на латентную функцию f , которая затем сжимается через функцию связи для получения вероятностной классификации. Латентная функция f - это так называемая “шумовая” функция, значения которой не наблюдаются и сами по себе не являются значимыми. Ее назначение - позволить удобно сформулировать модель, и f удаляется (интегрируется) во время предсказания. GaussianProcessClassifier реализует логистическую функцию связи, для которой интеграл не может быть вычислен аналитически, но легко аппроксимируется в бинарном случае.

В отличие от регрессии, апостериор латентной функции f не является гауссовым даже для GP-приор, поскольку гауссово правдоподобие неуместно для дискретных меток классов. Вместо этого используется негауссово правдоподобие, соответствующее логистической функции связи (logit).

GaussianProcessClassifier аппроксимирует негауссовский апостериор гауссовкой, основанной на аппроксимации Лапласа. Более подробную информацию можно найти в главе 3 [\[RW2006\]](#).

Предполагается, что среднее значение GP приора равно нулю. Ковариация приора задается путем передачи объекта [ядра](#).

Гиперпараметры ядра оптимизируются во время обучения GaussianProcessRegressor путем максимизации лог-маргинального правдоподобия (log-marginal-likelihood - LML) на основе переданного [optimizer](#). Поскольку LML может иметь несколько локальных оптимумов, оптимизатор можно запускать многократно, указав [n_restarts_optimizer](#). Первый запуск всегда выполняется с начальными значениями гиперпараметров ядра; последующие запуски выполняются со значениями гиперпараметров, выбранными случайным образом из диапазона допустимых значений.

Если начальные гиперпараметры должны быть фиксированными, в качестве оптимизатора можно передать [None](#).

[GaussianProcessClassifier](#) поддерживает многоклассовую классификацию, выполняя обучение и предсказание по принципу “один против всех” или “один против одного”. В режиме “один против всех” для каждого класса устанавливается один бинарный классификатор гауссовых процессов, который обучается отделять этот класс от остальных. В режиме “один против одного” для каждой пары классов подбирается один бинарный классификатор гауссовых процессов, который обучается разделять эти два

класса. Предсказания этих бинарных предикторов объединяются в многоклассовые предсказания. Более подробную информацию см. в разделе [многоклассовая классификация](#).

В случае классификации по гауссовскому процессу “один против одного” может быть вычислительно дешевле, поскольку приходится решать множество задач, затрагивающих только подмножество всего обучающего набора, а не меньшее количество задач на всем наборе данных. Поскольку классификация с помощью гауссовского процесса масштабируется кубически с размером набора данных, это может быть значительно быстрее. Однако обратите внимание, что “один против одного” не поддерживает предсказание оценок вероятности, а только простые предсказания. Более того, обратите внимание, что [GaussianProcessClassifier](#) не реализует (пока) истинную многоклассовую аппроксимацию Лапласа внутри, но, как обсуждалось выше, основан на решении нескольких бинарных задач классификации внутри, которые объединяются с помощью “один против всех” или “один против одного”.

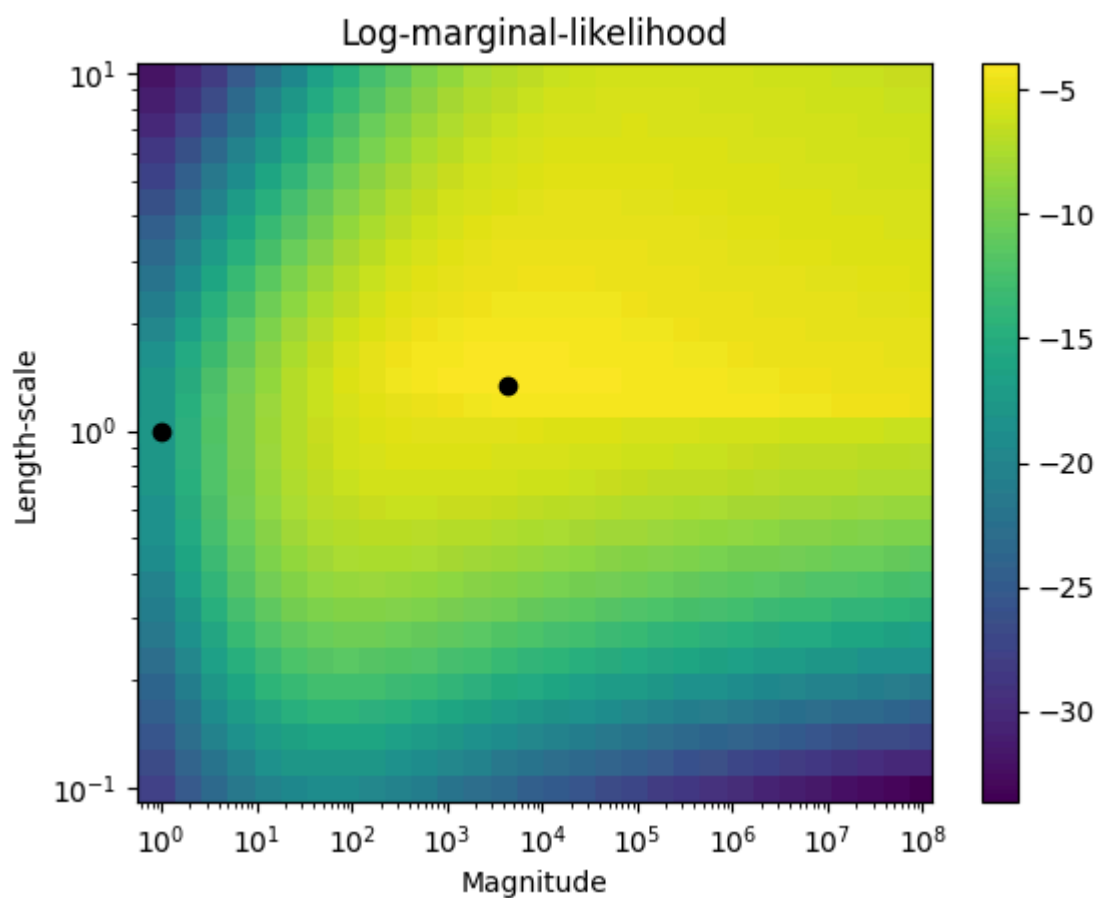
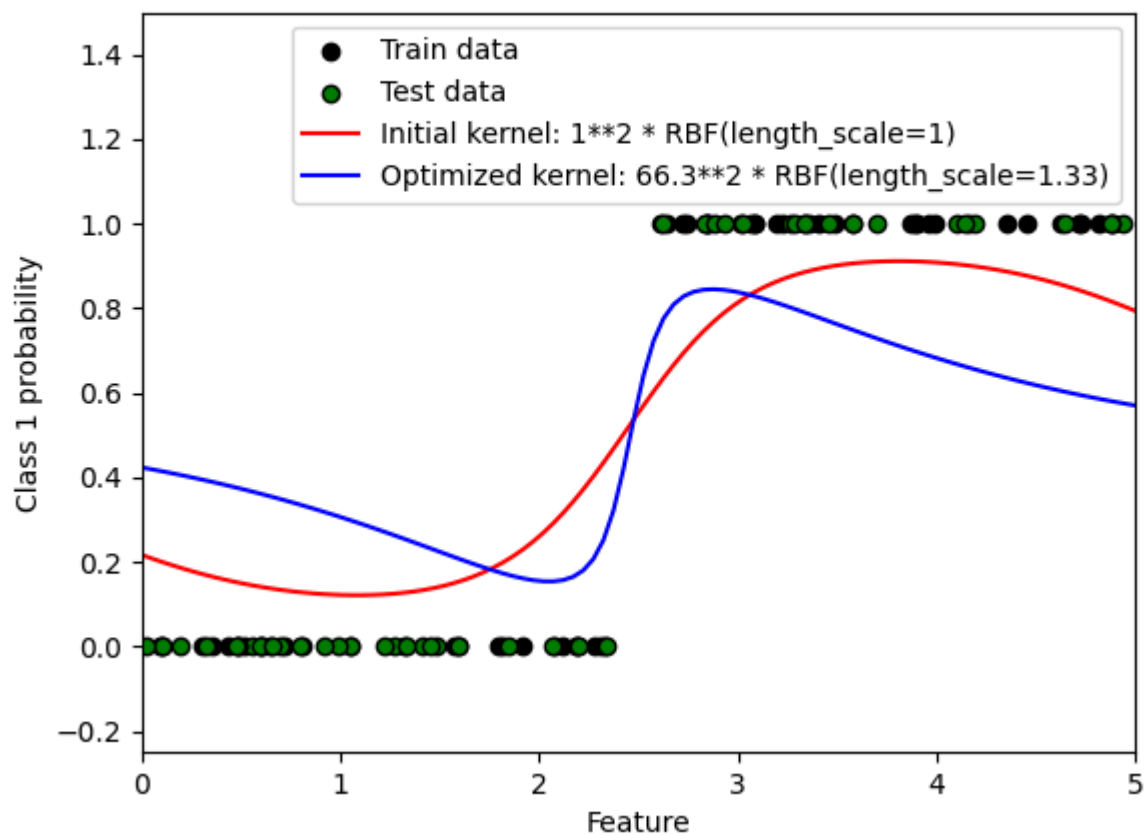
1.7.3. Примеры GPC

1.7.3.1. Вероятностные прогнозы с помощью GPC

Этот пример иллюстрирует предсказание вероятности GPC для ядра RBF с различными вариантами гиперпараметров. На первом рисунке показана предсказанная вероятность GPC с произвольно выбранными гиперпараметрами и с гиперпараметрами, соответствующими максимальному лог-маргинальному правдоподобию (LML).

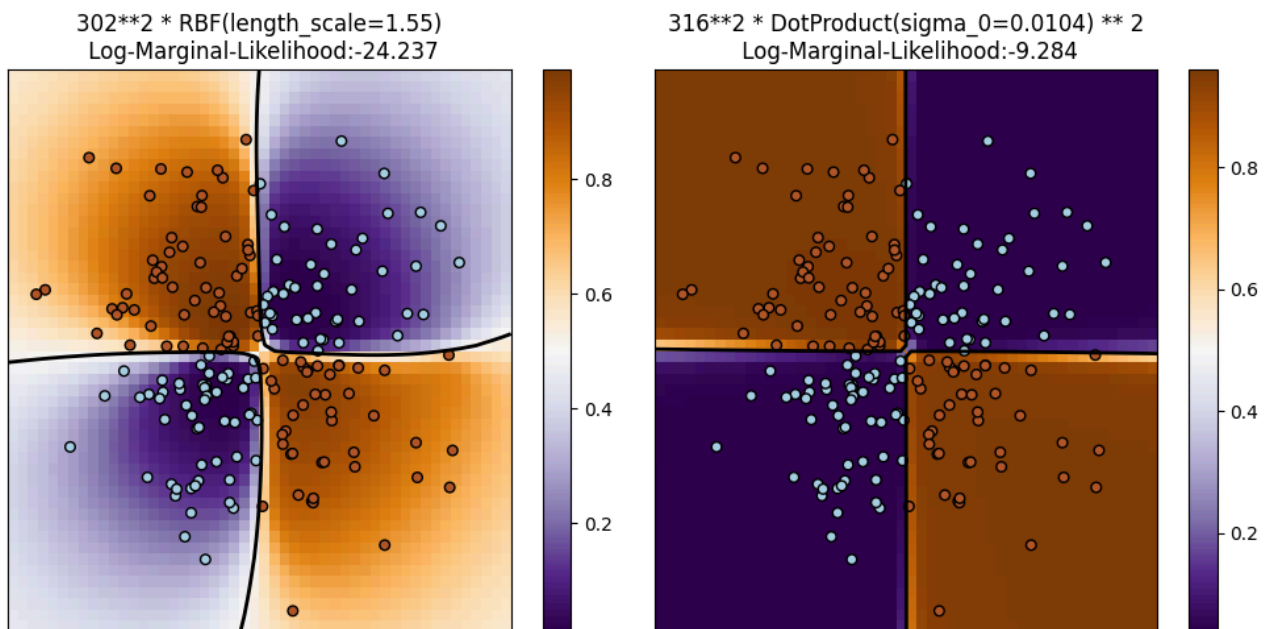
Хотя гиперпараметры, выбранные путем оптимизации LML, имеют значительно большее LML, они работают немного хуже в соответствии с log-loss на тестовых данных. На рисунке видно, что это происходит потому, что они демонстрируют крутое изменение вероятностей классов на границах классов (что хорошо), но имеют предсказанные вероятности, близкие к 0.5 вдали от границ классов (что плохо). Этот нежелательный эффект вызван аппроксимацией Лапласа, используемой внутри GPC.

На втором рисунке показано лог-маргинальное правдоподобие для различных вариантов гиперпараметров ядра, черными точками выделены два варианта гиперпараметров, использованных на первом рисунке.



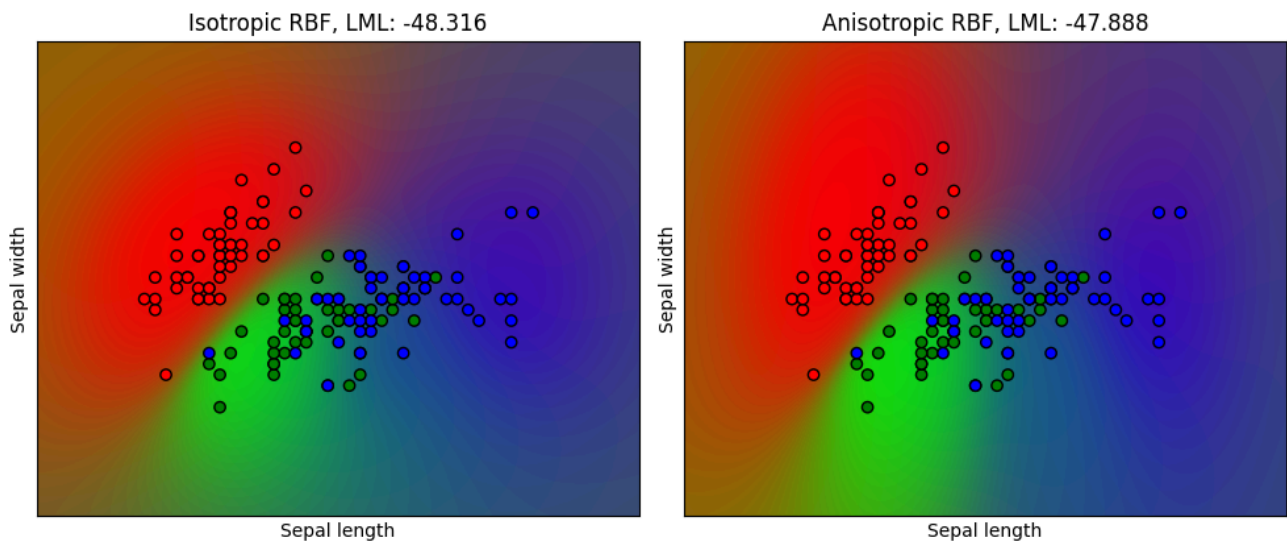
1.7.3.2. Иллюстрация GPC на наборе данных XOR

Этот пример иллюстрирует GPC на данных XOR. Сравниваются стационарное, изотропное ядро ([RBF](#)) и нестационарное ядро ([DotProduct](#)). На данном конкретном наборе данных ядро [DotProduct](#) дает значительно лучшие результаты, поскольку границы классов линейны и совпадают с осями координат. Однако на практике стационарные ядра, такие как [RBF](#) , часто дают лучшие результаты.



1.7.3.3. Классификация с помощью гауссовского процесса (GPC) на наборе данных Ирис

Этот пример иллюстрирует предсказанную вероятность GPC для изотропного и анизотропного RBF-ядер на двумерной версии набора данных Ирис. Это иллюстрирует применимость GPC к небинарной классификации. Анизотропное RBF-ядро получает немного большее лог-маргинальное правдоподобие за счет различных шкал длины для двух измерений признаков.



1.7.4. Ядра для гауссовских процессов

Ядра (также называемые “ковариационными функциями” в контексте GP) являются важнейшим компонентом GP, определяющим форму предшествующего и последующего GP. Они кодируют предположения об обучаемой функции, определяя “схожесть” двух точек данных в сочетании с предположением, что похожие точки данных должны иметь похожие целевые значения. Можно выделить две категории ядер: стационарные ядра зависят только от расстояния между двумя точками данных, а не от их абсолютных значений $k(x_i, x_j) = k(d(x_i, x_j))$ и поэтому инвариантны к трансляциям во входном пространстве, а нестационарные ядра зависят также от конкретных значений точек данных. Стационарные ядра можно также разделить на изотропные и анизотропные, причем изотропные ядра также инвариантны к поворотам во входном пространстве. За более подробной информацией мы обращаемся к главе 4 работы [\[RW2006\]](#). За рекомендациями по оптимальному сочетанию различных ядер мы обращаемся к [\[Duv2014\]](#).

API ядра гауссова процесса.



1.7.4.1. Базовые ядра

Ядро [ConstantKernel](#) может использоваться как часть ядра [Product](#), где оно масштабирует величину другого коэффициента (ядра), или как часть ядра [Sum](#), где оно

изменяет среднее значение гауссовского процесса. Он зависит от параметра $\backslash(\text{constant_value})$. Он определяется как:

$$\backslash[k(x_i, x_j) = \text{constant_value } \backslash;\text{forall } x_1, x_2]$$

В основном ядро [WhiteKernel](#) используется как часть ядра суммы, где оно объясняет шумовую составляющую сигнала. Настройка его параметра $\backslash(\text{noise_level})$ соответствует оценке уровня шума. Он определяется как:

$$\backslash[k(x_i, x_j) = \text{noise_level } \backslash\text{if } x_i == x_j \backslash\text{else } 0]$$

1.7.4.2. Операторы ядра

Операторы ядра берут одно или два базовых ядра и объединяют их в новое ядро. Ядро

[Sum](#) берет два ядра $\backslash(k_1)$ и $\backslash(k_2)$ и объединяет их посредством

$\backslash(k_{\text{sum}}(X, Y) = k_1(X, Y) + k_2(X, Y)$. Ядро [Product](#) берет два ядра $\backslash(k_1)$ и $\backslash(k_2)$ и

комбинирует их через $\backslash(k_{\text{product}}(X, Y) = k_1(X, Y) * k_2(X, Y)$. Ядро [Exponentiation](#)

берет одно базовое ядро и скалярный параметр $\backslash(p)$ и комбинирует их через

$\backslash(k_{\text{exp}}(X, Y) = k(X, Y)^p$. Обратите внимание, что магические методы `__add__`,

`__mul__` и `__pow__` переопределяются на объектах ядра, поэтому можно использовать, например, `RBF() + RBF()` как сокращение для `Sum(RBF(), RBF())`.

1.7.4.3. Ядро радиальной базисной функции (Radial basis function - RBF)

Ядро [RBF](#) - это стационарное ядро. Оно также известно как “квадратичное

экспоненциальное” ядро. Оно параметризуется параметром масштаба длины $\backslash(l > 0)$,

который может быть либо скаляром (изотропный вариант ядра), либо вектором с тем же числом измерений, что и входные данные $\backslash(x)$ (анизотропный вариант ядра). Ядро

задается:

$$\backslash[k(x_i, x_j) = \text{exp}\backslash\left(- \frac{d(x_i, x_j)^2}{2l^2} \right)$$

где $\backslash(d(\cdot, \cdot))$ - евклидово расстояние. Это ядро бесконечно дифференцируемо,

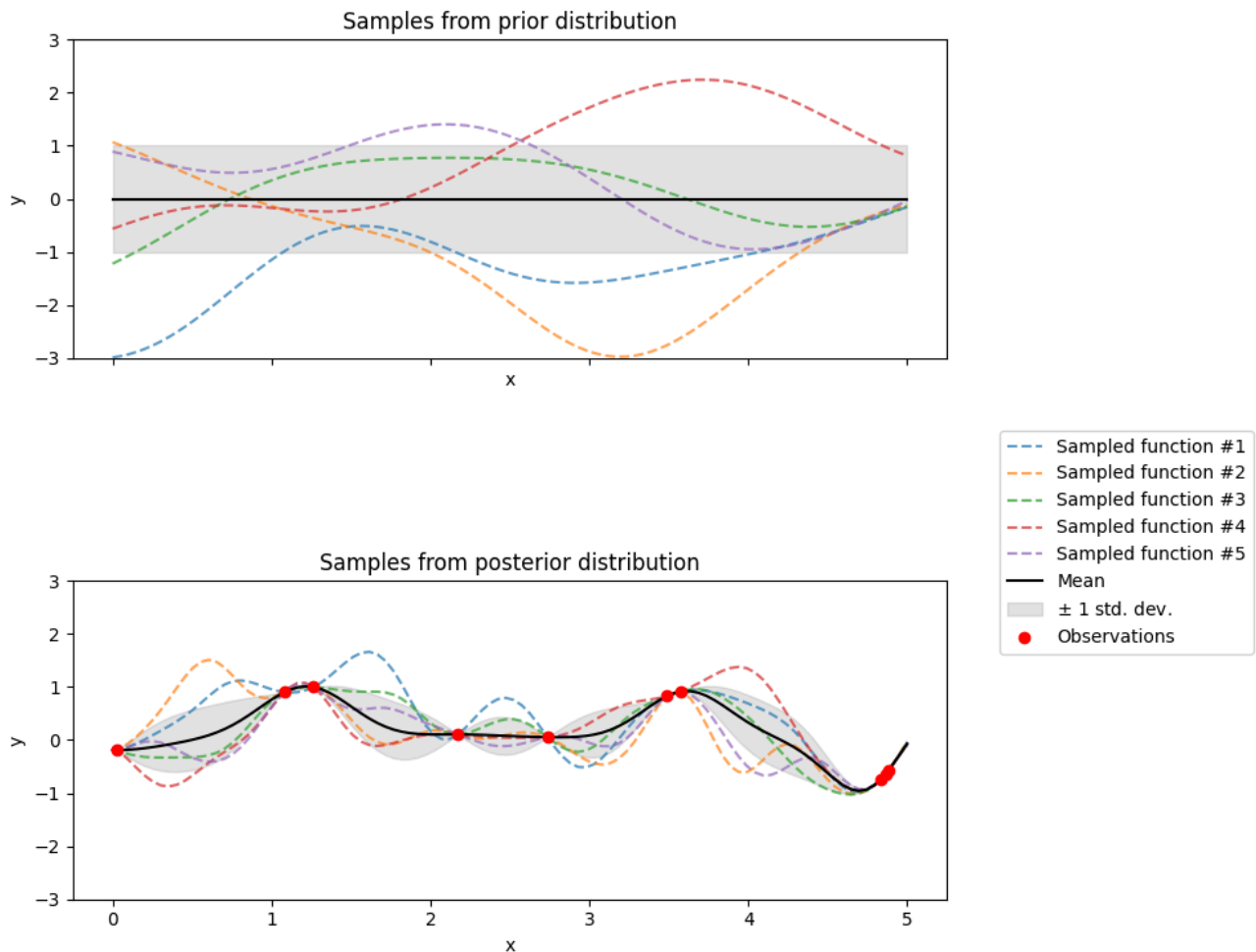
что означает, что GP с этим ядром в качестве ковариационной функции имеют

среднеквадратичные производные всех порядков и, таким образом, являются очень

гладкими. Предварительное и последующее значения GP, полученного с помощью ядра

RBF, показаны на следующем рисунке:

Radial Basis Function kernel

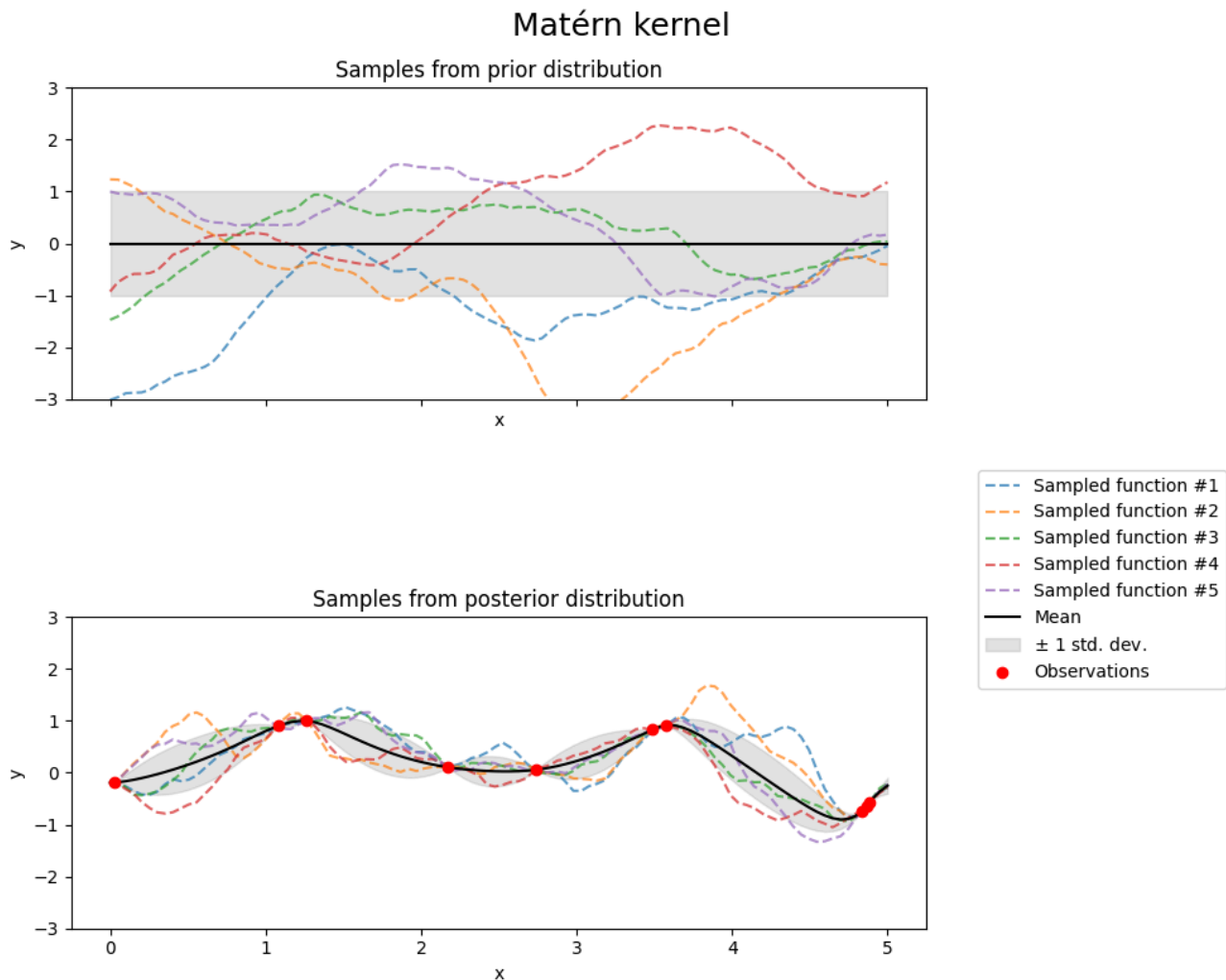


1.7.4.4. Ядро Матерна

Ядро [Matern](#) - это стационарное ядро и обобщение ядра [RBF](#). Оно имеет дополнительный параметр ν , который управляет гладкостью результирующей функции. Параметризуется параметром масштаба длины $l > 0$, который может быть либо скаляром (изотропный вариант ядра), либо вектором с тем же числом измерений, что и входные данные x (анизотропный вариант ядра).

Математическая реализация ядра Матерна.

Приор и апостериор GP, полученного с помощью ядра Матерна, показаны на следующем рисунке:



Более подробно о различных вариантах ядра Матерна см. в [\[RW2006\]](#), стр. 84.

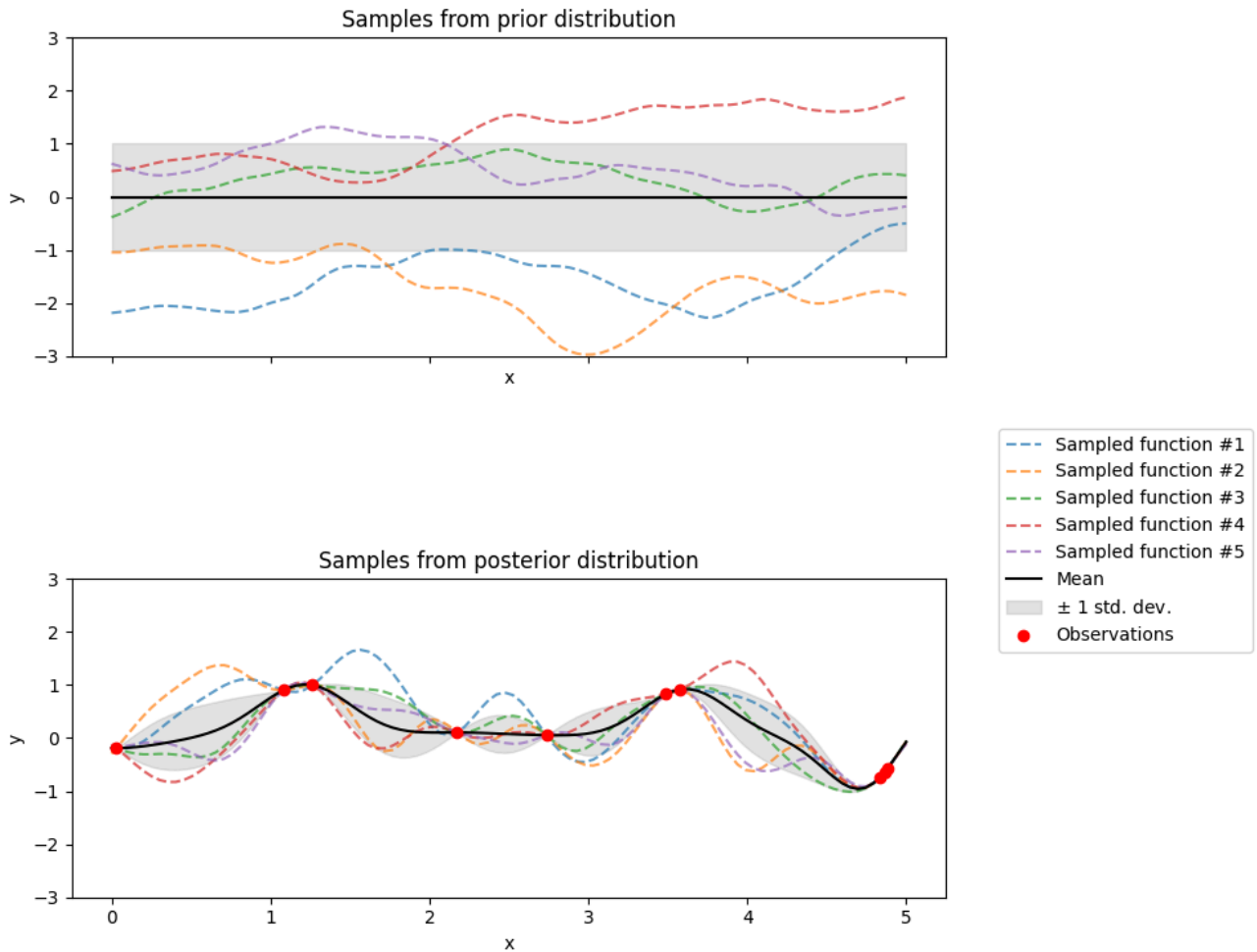
1.7.4.5. Рациональное квадратичное ядро

Ядро [RationalQuadratic](#) можно рассматривать как смесь масштабов (бесконечную сумму) ядер [RBF](#) с различными характерными масштабами длины. Параметризуется параметром масштаба длины $(l > 0)$ и параметром смеси масштабов $(\alpha > 0)$. На данный момент поддерживается только изотропный вариант, где (l) - скаляр. Ядро задается:

$$k(x_i, x_j) = \left(1 + \frac{d(x_i, x_j)^2}{2\alpha l^2}\right)^{-\alpha}$$

Приор и апостериор GP, полученного с помощью ядра [RationalQuadratic](#), показаны на следующем рисунке:

Rational Quadratic kernel



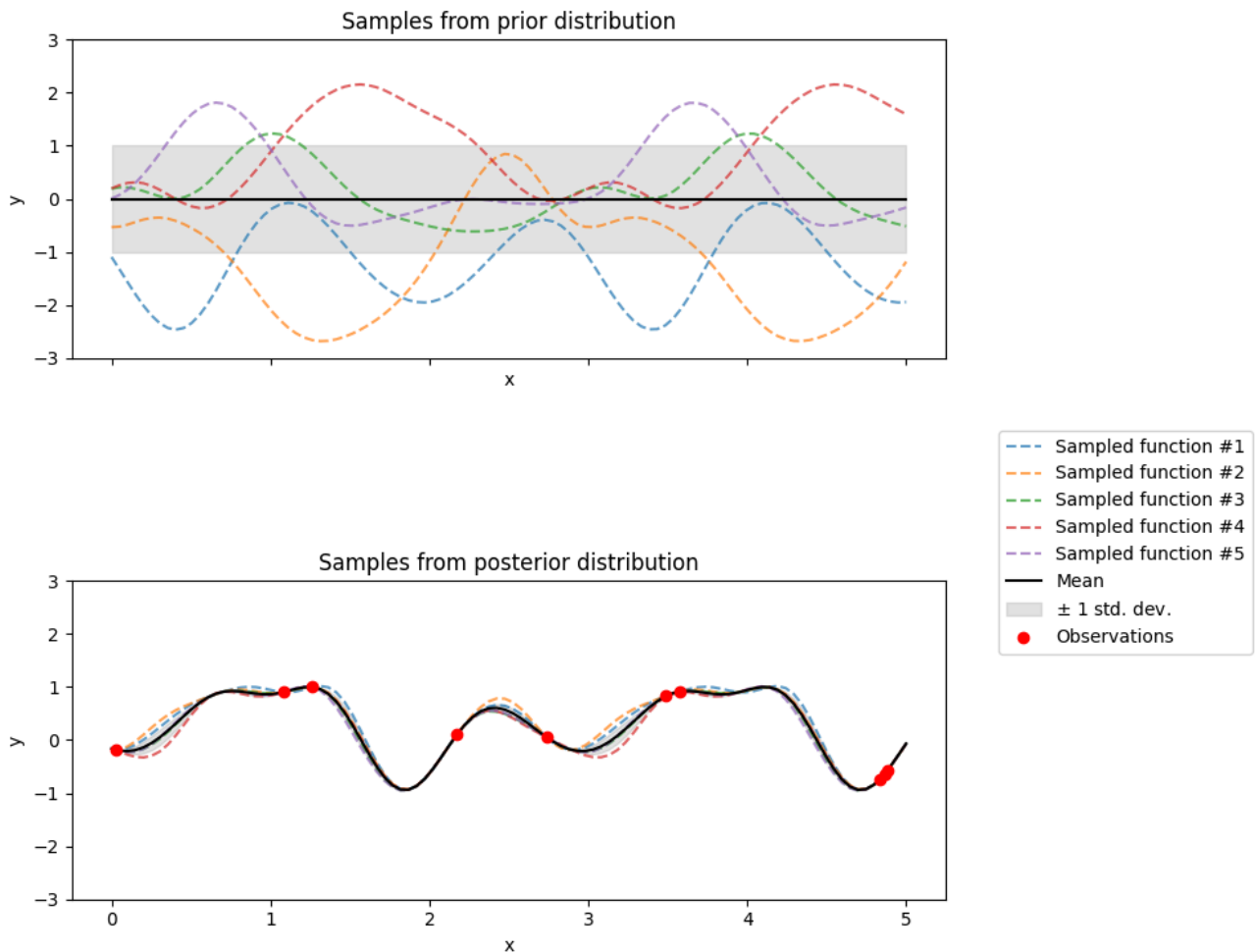
1.7.4.6. Ядро Exp-Sine-Squared

Ядро [ExpSineSquared](#) позволяет моделировать периодические функции. Оно параметризуется параметром масштаба длины $(l > 0)$ и параметром периодичности $(p > 0)$. На данный момент поддерживается только изотропный вариант, где (l) - скаляр. Ядро задается:

$$k(x_i, x_j) = \exp\left(-\frac{2 \sin^2(\pi d(x_i, x_j) / p)}{l^2}\right)$$

Приор и апостериор GP, полученного с помощью ядра ExpSineSquared, показаны на следующем рисунке:

Exp-Sine-Squared kernel



1.7.4.7. Ядро Dot-Product

Ядро [DotProduct](#) является нестационарным и может быть получено из линейной регрессии путем наложения $\mathcal{N}(0, 1)$ приор на коэффициенты x_d ($d = 1, \dots, D$) и приор $\mathcal{N}(0, \sigma_0^2)$ на смещение. Ядро [DotProduct](#) инвариантно к повороту

координат относительно начала координат, но не к трансляции. Параметризуется параметром σ_0^2 . Если $\sigma_0^2 = 0$, то ядро называется однородным линейным ядром, в противном случае - неоднородным. Ядро задается следующим образом

Данный сайт является неофициальным переводом сайта scikit-learn.org

Если вы хотите помочь проекту с переводом, то можно обращаться по следующему адресу support@scikit-learn.ru